ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ № 10

Թեմա՝ Դինամիկ անդամներ և լռելյայն կոնստրուկտոր

Ուսանող՝ Գաբրիել Քոչարյան

1. ԱՇԽԱՏԱՆՔԻ ՆՊԱՏԱԿԸ ԵՒ ՊԱՀԱՆՋՆԵՐԸ

Նպատակը: Ստեղծել դատարկ և տվյալներով օբյեկտներ, անվտանգ փոփոխել դինամիկ զանգվածի չափը, իրականացնել պահանջվող հաշվարկները:

Ընդհանուր պահանջներ: Դասում պահել `private double*` զանգված, n չափ և p պարամետր: Լռելյայն կոնստրուկտորը ստեղծում է դատարկ օբյեկտ (`n=0, data=nullptr`): Ավելացնել պարամետրերով կոնստրուկտոր, դեստրուկտոր, `append(value)`, `removeAt(index)`, `print()`: Չափի փոփոխությունը կատարել նոր memory հատկացնելով, պատճենելով և հինն ազատելով:

2. ԱՆՀԱՏԱԿԱՆ ԱՌԱՋԱԴՐԱՆՔ (№ 17 — ՓՈԽԱՆՑՈՒՄՆԵՐԻ ՄԻՋՆՈՐԴԱՎՃԱՐՆԵՐ)

Տվյալներ: Ստեղծել `TransferRegister` դաս՝ պահելով փոխանցումների գումարների զանգվածը և p միջնորդավճարի տոկոսը (`fee = amount · p / 100`):

Հաշվարկներ: 1) Հաշվել միջնորդավճարների ընդհանուր գումարը, 2) Գտնել ստացողին հասնող ամենափոքր գումարով փոխանցման ինդեքսը (`net = amount - fee`), 3) Հաշվել `fee > 1000` պայմանով փոխանցումների քանակը:

Թույլատրելի արժեքներ: Բոլոր գումարները `> 0`, `0 ≤ p ≤ 100`: Լռելյայն `p = 0`: Փորձնական տվյալներ: `amount = {10000, 50000, 120000, 200000}`, `p = 1`:

3. C++ ԾՐԱԳՐԱՅԻՆ ԿՈԴ (MAIN.CPP)

```
#include <iostream>
#include <iomanip>
#include <cmath>

class TransferRegister {
private:
    double* data;
    int n;
    double p;

    bool isValidAmount(double val) const { return std::isfinite(val) && val > 0; }
    bool isValidP(double p_val) const { return std::isfinite(p_val) && p_val >= 0 && p_val <= 100; }

public:
    TransferRegister(double p_val = 0.0) : data(nullptr), n(0), p(0.0) {
        if (isValidP(p_val)) p = p_val;
    }

    TransferRegister(const double* arr, int count, double p_val) : data(nullptr), n(0), p(0.0) {
        if (!isValidP(p_val) || count < 3 || count > 30) return;
        for (int i = 0; i < count; ++i) if (!isValidAmount(arr[i])) return;
        p = p_val; n = count; data = new double[n];
        for (int i = 0; i < n; ++i) data[i] = arr[i];
    }

    ~TransferRegister() { delete[] data; }
    TransferRegister(const TransferRegister&) = delete;
    TransferRegister& operator=(const TransferRegister&) = delete;

    bool append(double value) {
        if (!isValidAmount(value) || n >= 30) return false;
        double* newData = new double[n + 1];
        for (int i = 0; i < n; ++i) newData[i] = data[i];
        newData[n] = value;
        delete[] data; data = newData; n++;
        return true;
    }

    bool removeAt(int index) {
        if (index < 0 || index >= n) return false;
        if (n - 1 == 0) { delete[] data; data = nullptr; n = 0; return true; }
        double* newData = new double[n - 1];
        for (int i = 0, j = 0; i < n; ++i) if (i != index) newData[j++] = data[i];
        delete[] data; data = newData; n--;
        return true;
    }

    double totalFee() const {
        if (n == 0) return 0.0;
        double sum = 0.0;
        for (int i = 0; i < n; ++i) sum += data[i] * (p / 100.0);
        return sum;
    }

    int minIndexNet() const {
        if (n == 0) return -1;
        int minIdx = 0;
        double minNet = data[0] - (data[0] * p / 100.0);
        for (int i = 1; i < n; ++i) {
            double net = data[i] - (data[i] * p / 100.0);
            if (net < minNet - 1e-9) { minNet = net; minIdx = i; }
        }
        return minIdx;
    }

    int countHighFee() const {
        if (n == 0) return 0;
        int count = 0;
        for (int i = 0; i < n; ++i) if ((data[i] * p / 100.0) > 1000.0 + 1e-9) count++;
        return count;
    }
```

4. ՓՈՐՁԱՐԿՄԱՆ ԱՐԴՅՈՒՆՔՆԵՐ ԵՒ ՍՏՈՒԳԻՉ ՀԱՇՎԱՐԿ

Օբյեկտ	Ձանգված (amount)	p (%)	Ընդհանուր fee	Մին. net ինդեքս	fee > 1000 քանակ
Obj1 (Հեռացված idx 1)	{10000, 120000, 200000}	0%	0.00	0 (9900)	0
Obj2 (Սկզբնական)	{10000, 50000, 120000, 200000}	1%	3800.00	0 (9900)	2

Ձեռքով հաշվարկ (Obj2): fee = {100, 500, 1200, 2000} → Գումար = 3800; net = {9900, 49500, 118800, 198000} → minIndex = 0; fee > 1000 (1200, 2000) → քանակ = 2:

5. ՍՏՈՒԳՄԱՆ ՀԱՐՑԵՐԻ ՊԱՏԱՍԽԱՆՆԵՐ

1. Ի՞նչ է լռելյայն կոնստրուկտորը:
Կոնստրուկտոր, որը կանչվում է առանց արգումենտների և օբյեկտը բերում է անվտանգ նախնական վիճակի (n=0, data=nullptr):

2. Ինչպե՞ս սկզբնաարժեքավորել դինամիկ անդամը:
Դատարկ վիճակում վերագրելով `nullptr`, իսկ տվյալների առկայության դեպքում՝ `new double[n]` հրամանով:

3. Ինչո՞ւ պետք է ճիշտ ազատել հատկացված հիշողությունը:
Հիշողության արտահոսքերից (memory leaks) խուսափելու համար. dynamic allocation-ից հետո հին memory-ն պետք է ջնջել `delete[]`-ով:

4. Ինչպե՞ս ստուգել դատարկ կամ սխալ սկզբնական տվյալները:
Ստուգելով `n == 0` կամ ցուցչի `nullptr` լինելը, ինչպես նաև ստուգելով տվյալների թույլատրելի տիրույթը (amount > 0, 0 ≤ p ≤ 100):

6. ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ

Աշխատանքի ընթացքում ուսումնասիրվեց C++ դասերում դինամիկ հիշողության կառավարման սկզբունքները, իրականացվեցին ռեսուրսների անվտանգ հատկացման և ազատման մեթոդներ (RAII concept), ինչպես նաև հաջողությամբ ստուգվեցին `TransferRegister` դասի հաշվարկային արդյունքները: